



**SHAHEED HIMAYUN MUZZAMIL MEMORIAL
GOVERNMENT DEGREE COLLEGE ANANTNAG**

Department of Computer Applications

**Project Report (e-yantra)
Time-Based Grid Localization Robot**

Batch 2022 (NEP)

Mohd Numan Asgar	65301-ANG-2022
Ubaid Mushtaq Mir	65291-ANG-2022
Wasiq Tariq	63848-ANG-2021
Danish Ellahi	65277-ANG-2022
Aqib Javid Bhat	65274-ANG-2022
Ifham Mushtaq	65292-ANG-2022

Under the Supervision of
Dr. Tawseef Ahmed Teli
Asst. Prof. Department of Computer Applications

Contents

Abstract	2
1. Introduction.....	2
1.1 Background and Motivation.....	2
1.2 Problem Statement	2
1.3 Scope and Applications.....	2
2. Methodology.....	3
2.1 Working Principle	3
2.2 Hardware Components Used.....	3
2.2.1 Arduino UNO	3
2.2.2 L298N Motor Driver.....	4
2.2.3 Dual Shaft BO Motors	4
2.2.4 65mm BO Motor Wheels	5
2.2.5 Chassis.....	5
2.2.6 Jumper Wires	5
2.2.7 4AA Battery Holder.....	6
2.2.8 AA Batteries	6
2.2.9 ON/OFF Switch.....	7
2.2.10 170-Ponit Mini Breadboard.....	7
2.3 Software and Algorithms Used.....	7
2.3.1 Programming Language and Platform Used.....	7
2.3.2 Algorithm and Pseudocode	7
2.3.3 Flowchart.....	9
2.3.5 Code	10
2.4 Circuit Diagram.....	14
3. Implementation.....	14
3.1 Build and Assembly	14
3.2 Challenges faced and how they were resolved.....	15
4. Testing and Results.....	16
4.1 Tests Performed.....	16
4.2 Observations and Outcomes	17
5. Conclusion and Future Scope.....	18
6. GitHub Repository.....	18

Abstract

In the domain of autonomous systems, precise navigation within structured environments is a cornerstone of automation. This project presents the design, implementation, and evaluation of a Time-Based Grid Localization Robot. This robot demonstrates a fundamental navigation technique known as dead reckoning, where it determines its position by executing a sequence of pre-calculated movements without relying on external sensors for feedback. The system is designed around an Arduino Uno microcontroller, which serves as the central processing unit. It navigates a 5x5 grid, represented internally as a 2D array, to reach a target coordinate. Motor control is managed by an L298N driver, which powers a 4-wheeled drive system. The robot's localization is achieved by running its motors for precisely calibrated durations of time to traverse grid cells and perform 90-degree turns. This project provides a practical exploration of open-loop control systems, highlighting their effectiveness in predictable environments while also exposing their inherent vulnerabilities to cumulative error and external disturbances. The successful implementation of this robot serves as a foundational prototype for understanding algorithmic navigation, motor control precision, and the principles of localization in the absence of real-time sensory data.

1. Introduction

1.1 Background and Motivation

The rapid growth of robotics in industries such as logistics, manufacturing, and healthcare has created a demand for efficient and reliable autonomous navigation systems. While advanced robots utilize complex sensors such as LiDAR and vision systems for Simultaneous Localization and Mapping (SLAM), many real-world applications occur in highly predictable, grid-like environments, such as warehouses or assembly lines. In these scenarios, simpler, more cost-effective navigation strategies can be deployed. This project was motivated by the desire to explore one of the most fundamental of these strategies, known as *Dead Reckoning*¹. By building a robot that navigates using only an internal map and timed movements, the aim is to gain a deep, practical understanding of the core principles and challenges of autonomous control.

1.2 Problem Statement

In many automated environments, the cost and complexity of advanced sensor suites are prohibitive for simple, repetitive tasks. The central problem this project addresses is the development of a low-cost, autonomous robot capable of navigating from a known origin to a specified destination on a 2D grid without any external feedback sensors. The navigation must be entirely self-contained, relying on a pre-programmed map and a highly accurate, time-based execution of movements to control its position and orientation.

1.3 Scope and Applications

The primary scope of this project is the construction and programming of a 4WD robot

¹ *Dead reckoning is a method used to estimate the current position of an object by using its last known location and information about its velocity or travelled distance*

that can autonomously find and travel to a target coordinate on a 5x5 grid. The robot's path-planning logic is designed to first align with the destination's Y-axis, traverse the required vertical distance, turn 90 degrees, and then traverse the horizontal distance along the X-axis. Despite its operational simplicity, this technology forms the basis for numerous applications. For instance, it is integral to Automated Guided Vehicles (AGVs) that transport materials along fixed routes in factories and distribution centers. The technology also serves as a foundation for educational platforms used to teach fundamental concepts in programming, kinematics, and open-loop control systems. Furthermore, it is employed in laboratory automation for moving samples or equipment between predefined stations in a controlled lab environment.

2. Methodology

2.1 Working Principle

The robot operates on the principle of an open-loop control system, where its actions are not based on real-time feedback from the environment but on a predetermined sequence. The entire logic is orchestrated by the Arduino Uno microcontroller. The process begins at startup. The `setup()` function configures the digital pins connected to the L298N motor driver as outputs. Immediately after, it calls the `get_destination_coordinates()` function, which iterates through the hardcoded GRID (a 5x5 2D array) to locate the cell containing the value 1. The row and column indices of this cell are stored as the target coordinates, `destX` and `destY`. The main operational logic resides in the `loop()` function, which executes the navigation sequence once. It calls `gotoDestination(destX, destY)`, which first calculates the required travel distances, `deltaX` and `deltaY`, by subtracting the robot's current coordinates (initially 0,0) from the destination coordinates.

The movement is executed in a strict, non-optimized sequence. Initially, the robot performs vertical travel along the Y-axis, moving forward for a duration calculated by `deltaY * CELL_TRAVERSE_TIME`, where `CELL_TRAVERSE_TIME` is a pre-calibrated constant. After coming to a stop, it executes a reorientation with a 90-degree turn to face the correct direction along the X-axis, a process that takes a fixed duration of `TURN_TIME_90`. The robot then begins its horizontal travel, moving forward again for a time determined by `abs(deltaX) * CELL_TRAVERSE_TIME`. Upon reaching its destination, the robot enters its final state by updating its internal coordinates (`curX`, `curY`) and performing one last turn to reset its orientation to face forward along the Y-axis. The accuracy of this entire process is critically dependent on these pre-calibrated time constants for movement and turning, as there are no sensors to correct for any deviations from the intended path.

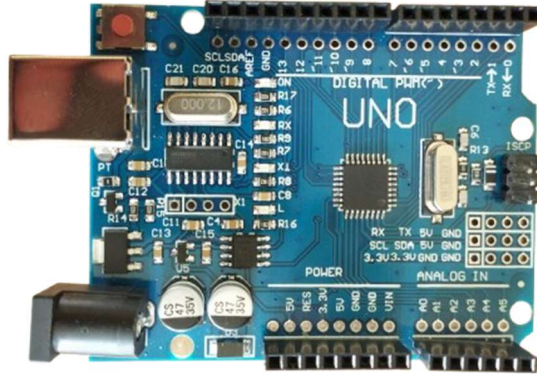
2.2 Hardware Components Used

This section describes the electronic and mechanical components used in building the robot, along with their roles and technical specifications.

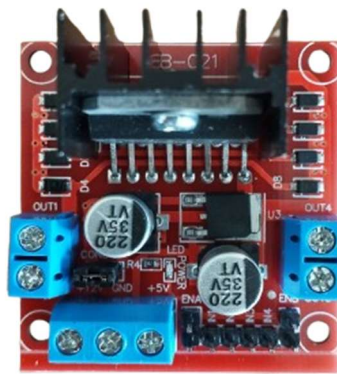
2.2.1 Arduino UNO

The Arduino Uno is a microcontroller board based on the ATmega328P. It features 14 digital input/output pins (of which 6 can be used as PWM outputs), 6 analog inputs, a 16 MHz quartz crystal, USB connection, power jack, ICSP header, and a reset button. It operates at 5V and can

be powered via USB or external power supply. In this project, the Arduino Uno acts as the central control unit. It is responsible for storing the grid map, executing the algorithm, calculating the required movement durations, and sending precise control signals to the L298N motor driver to direct the robot's motion.



2.2.2 L298N Motor Driver



The L298N motor driver module is a dual H-bridge driver that can control two DC motors independently. It can handle up to 2A of current per channel and supports voltage inputs from 5V to 12V. It has onboard terminal blocks for motors and power, and input pins for control signals from the Arduino. The motor driver acts as an interface between the low-power control signals from the Arduino and the high-current demands of the motors. It allows the Arduino to control the speed and direction of each motor, enabling forward, backward, and turning movements of the robot.

2.2.3 Dual Shaft BO Motors

Each BO motor operates on 3V–12V DC and provides moderate torque with low RPM, typically around 100 RPM at 6V. The dual shaft allows for attachments on both ends, useful for future upgrades like encoders for speed sensing. These motors drive the robot's wheels and are responsible for its mobility. Their torque is suitable for lightweight robotic platforms, allowing smooth motion over flat indoor surfaces. The dual-shaft design also offers flexibility for expansion.

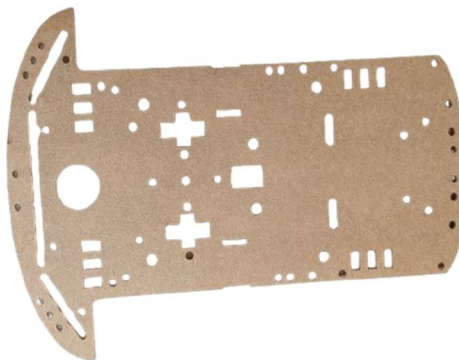


2.2.4 65mm BO Motor Wheels



These wheels have a diameter of 65mm and are made of lightweight plastic with a rubber outer ring for better grip. They are designed to be press-fitted onto BO motor shafts. The wheels enable the robot to move forward, backward, and turn efficiently. Their size provides a good balance between speed and stability, making them suitable for grid navigating applications.

2.2.5 Chassis

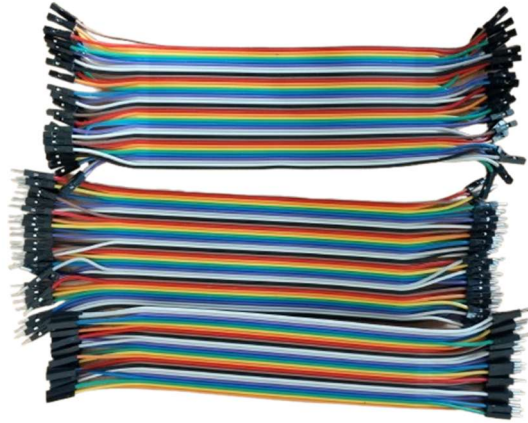


The chassis is typically made from acrylic or plastic and includes mounting holes for motors, battery holders, and control boards. It is lightweight, compact, and designed for BO motor compatibility. The chassis provides the structural base for mounting all the components. It organizes the robot into a stable form factor and ensures that everything stays securely in place during movement and testing.

2.2.6 Jumper Wires

Jumper wires are insulated electrical wires with connector pins at each end. They come in various types such as male-to-male, male-to-female, and female-to-female, and are typically used on breadboards and

headers. These wires are available in different lengths and colors for easy circuit tracing. Jumper wires are used for making electrical connections between the Arduino, motor driver, and breadboard. They enable flexible and temporary wiring, allowing quick prototyping and easy troubleshooting without the need for soldering. They also help in organizing the connections and keeping the circuit modular.



2.2.7 4AA Battery Holder



The battery holder accommodates four AA batteries and provides a combined output of approximately 12V. It supplies power to the motors and possibly the Arduino via the motor driver.

2.2.8 AA Batteries



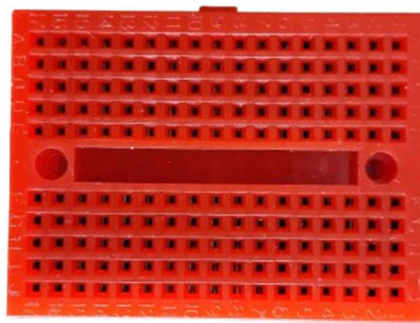
In this project, AA Li-Po rechargeable batteries are used that provide approx 3.5V each. Four batteries together supply around 12V. These batteries power the L298N motor driver, which in turn powers the Arduino, motors and other circuits of the robot. Using battery power makes the system portable and suitable for real-world navigation, free from wired power sources.

2.2.9 ON/OFF Switch



The On/Off switch is a simple SPST (Single Pole Single Throw) toggle switch. It usually has two terminals and is designed to interrupt or complete a circuit, allowing current to flow only when in the “ON” position. The switch is used to conveniently power the robot on or off without disconnecting the batteries. It adds a layer of safety and ease of use by allowing quick shutdowns during testing or in case of malfunction and helps conserve battery life when the robot is idle.

2.2.10 170-Pin Mini Breadboard



This is a compact solderless breadboard with 170 tie-points. It is made of ABS plastic and features two power rails along with connecting strips for temporary circuit construction. The breadboard is used to connect components like sensors, modules, and wires during the prototyping phase. It helps in testing and modifying the circuit quickly without soldering, which is ideal for student projects.

2.3 Software and Algorithms Used

This section outlines the programming tools, logic, and flow that control the behavior of the robot.

2.3.1 Programming Language and Platform Used

The entire robot is programmed using the **Arduino IDE**, which is a user-friendly open-source platform for writing and uploading code to the Arduino Uno. The language used is **C/C++**, specifically tailored for embedded systems programming. The Arduino IDE offers built-in functions for controlling digital and analog pins, reading sensor data, and communicating with external devices, which made it easy to implement the logic required for this project.

2.3.2 Algorithm and Pseudocode

The algorithm for the Time-Based Grid Localization Robot enables autonomous navigation to a predefined

destination on a 5x5 grid using time-based movement control. It identifies the destination marked on the grid and calculates the required steps along X and Y axes from the current position. Based on this delta, it moves forward, reverses, and rotates with predefined delays. The robot follows a deterministic route, making it suitable for structured environments like indoor grids or arenas without real-time obstacle dynamics.

2.3.2.1 Algorithm

1. Configure all motor control pins as outputs.
2. Programmatically scan the GRID 2D array to find the coordinates (destX, destY) of the cell marked with 1 store its coordinates as (destX, destY).
3. Calculate the number of cells to travel vertically ($\text{deltaY} = \text{destY} - \text{curY}$) and horizontally ($\text{deltaX} = \text{destX} - \text{curX}$).
4. Move the robot forward for a time equal to $\text{deltaY} * \text{CELL_TRAVERSE_TIME}$ ms to reach correct Y-coordinate. Update curY.
5. Stop the robot.
 - a. If deltaX is positive, turn the robot 90 degrees to the right.
 - b. If deltaX is negative, turn the robot 90 degrees to the left.
6. Stop. Move the robot forward for a time equal to $\text{abs}(\text{deltaX}) * \text{CELL_TRAVERSE_TIME}$ to reach the destination X-coordinate. Update curX.
7. Stop. Execute a 90-degree turn in the opposite direction of the first turn to return the robot to its original forward-facing orientation.
8. Stop all motors and enter an idle state.

2.3.2.2 Pseudocode

```

BEGIN
  INITIALIZE motor_pins as OUTPUT
  (destX, destY) = FIND_COORDINATES_OF_VALUE(1, GRID)
  currentX = 0, currentY = 0
  has_run = FALSE
  WHILE TRUE
    IF has_run == FALSE THEN
      deltaY = destY - currentY
      deltaX = destX - currentX

      IF deltaY > 0 THEN
        MOVE_FORWARD(time = deltaY * CELL_TRAVERSE_TIME)
        currentY = currentY + deltaY
      ENDIF
      STOP_MOTORS(2000ms)
      IF deltaX > 0 THEN
        TURN_RIGHT(time = TURN_90_TIME)
      ELSE IF deltaX < 0 THEN
        TURN_LEFT(time = TURN_90_TIME)
      ENDIF
      STOP_MOTORS(500ms)
      IF deltaX != 0 THEN
        MOVE_FORWARD(time = absolute(deltaX) * CELL_TRAVERSE_TIME)

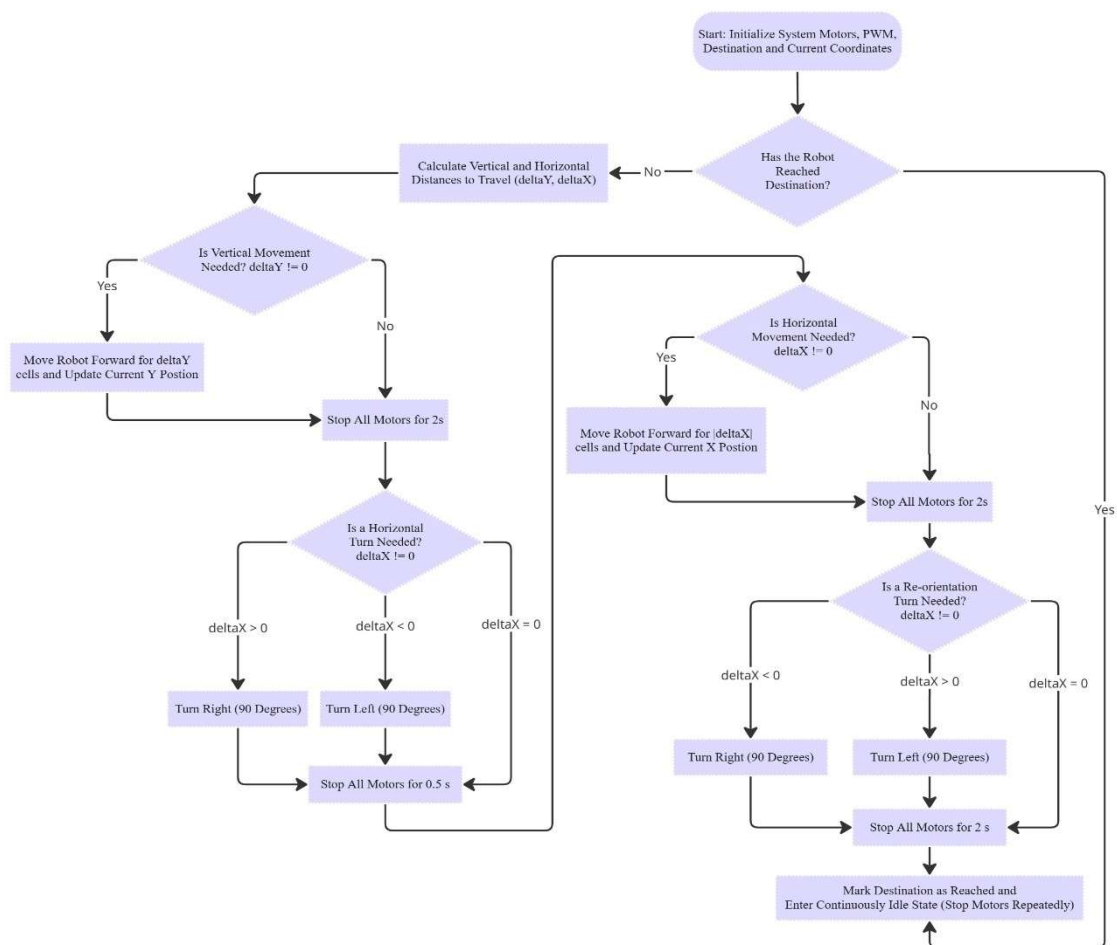
```

```

        currentX = currentX + deltaX
    ENDIF
    STOP_MOTORS(2000ms)
    IF deltaX > 0 THEN
        TURN_LEFT(time = TURN_90_TIME)
    ELSE IF deltaX < 0 THEN
        TURN_RIGHT(time = TURN_90_TIME)
    ENDIF
    STOP_MOTORS(2000ms)
    has_run = TRUE
ELSE
    STOP_MOTORS(2000ms)
ENDIF
ENDWHILE
END

```

2.3.3 Flowchart



2.3.5 Code

```
/*
 * TimeBasedGridLocalizationRobot.ino
 * An autonomous robot that navigates to a destination on a 5x5 grid using
 * time-based movement control.
 */

// Grid Configuration
#define ROWS 5
#define COLS 5

int GRID[ROWS][COLS] = {
    { 0, 0, 0, 0, 0 },
    { 0, 0, 0, 0, 0 },
    { 0, 0, 0, 0, 0 },
    { 0, 0, 0, 1, 0 }, // Destination marked as 1
    { 0, 0, 0, 0, 0 },
};

//Motor Driver Pins (PWM for motor speed control)
const int EN_A = 10; // Right motor enable pin (Grey)
const int EN_B = 11; // Left motor enable pin (Orange)

// Motors from each side are connected to the same output pins of Motor
// Driver
// Motor control pins
const int MOTOR_RIGHT_1 = 8; // INPUT 1 - Blue
const int MOTOR_RIGHT_2 = 9; // INPUT 2 - Purple
const int MOTOR_LEFT_1 = 6; // INPUT 3 - Green
const int MOTOR_LEFT_2 = 7; // INPUT 4 - Yellow

const int CELL_TRAVERSE_TIME = 880; // Time to traverse one grid cell
(ms)

// Time constants (in ms) for x-degree turn
enum RotationTime {
    TURN_TIME_180 = 1250, // Time for 180-degree turn
    TURN_TIME_90 = 890, // Time for 90-degree turn
};

// Robot position and destination tracking
int originX = 0;
int originY = 0;
int destX = 0;
int destY = 0;
int curX = 0;
```

```
int curY = 0;

int has_run = 0; // Flag to indicate destination is reached

void setup() {
    // Set motor driver front pins as OUTPUT pins
    pinMode(EN_A, OUTPUT);
    pinMode(EN_B, OUTPUT);
    pinMode(MOTOR_RIGHT_1, OUTPUT);
    pinMode(MOTOR_RIGHT_2, OUTPUT);
    pinMode(MOTOR_LEFT_1, OUTPUT);
    pinMode(MOTOR_LEFT_2, OUTPUT);

    // Find destination coordinates from the grid
    get_destination_coordinates();
}

void loop() {
    if (has_run == 0) {
        gotoDestination(destX, destY);
        has_run = 1; // Mark destination reached
    } else {
        stop(2000); // Stop once destination is reached
    }
}

// Get destination coordinates from the grid
void get_destination_coordinates() {
    for (int i = 0; i < ROWS; i++) {
        for (int j = 0; j < ROWS; j++) {
            if (GRID[i][j] == 1) { // Find destination cell marked with 1
                destX = i;
                destY = j;
                return;
            }
        }
    }
}

// Set speed for motors
void setSpeed(unsigned int speed) {
    analogWrite(EN_A, speed);
    analogWrite(EN_B, speed);
}

// Move forward all 4 motors for a given time and speed
void forward(unsigned int time, unsigned int speed) {
```

```
    setSpeed(speed);
    digitalWrite(MOTOR_RIGHT_1, HIGH);
    digitalWrite(MOTOR_RIGHT_2, LOW);
    digitalWrite(MOTOR_LEFT_1, HIGH);
    digitalWrite(MOTOR_LEFT_2, LOW);
    delay(time);
}

// Move backward all 4 motors for a given time and speed
void reverse(unsigned int time, unsigned int speed) {
    setSpeed(speed);
    digitalWrite(MOTOR_RIGHT_1, LOW);
    digitalWrite(MOTOR_RIGHT_2, HIGH);
    digitalWrite(MOTOR_LEFT_1, LOW);
    digitalWrite(MOTOR_LEFT_2, HIGH);
    delay(time);
}

// Stop all motors
void stop(unsigned int time) {
    digitalWrite(MOTOR_RIGHT_1, LOW);
    digitalWrite(MOTOR_RIGHT_2, LOW);
    digitalWrite(MOTOR_LEFT_1, LOW);
    digitalWrite(MOTOR_LEFT_2, LOW);
    delay(time);
}

// Turn robot left
void turnLeft(unsigned int time, unsigned int speed) {
    setSpeed(speed);
    digitalWrite(MOTOR_RIGHT_1, HIGH);
    digitalWrite(MOTOR_RIGHT_2, LOW);
    digitalWrite(MOTOR_LEFT_1, LOW);
    digitalWrite(MOTOR_LEFT_2, HIGH);
    delay(time);
}

// Turn robot right
void turnRight(unsigned int time, unsigned int speed) {
    setSpeed(speed);
    digitalWrite(MOTOR_RIGHT_1, LOW);
    digitalWrite(MOTOR_RIGHT_2, HIGH);
    digitalWrite(MOTOR_LEFT_1, HIGH);
    digitalWrite(MOTOR_LEFT_2, LOW);
    delay(time);
}
```

```
// Navigate to destination coordinates
void gotoDestination(int destX, int destY) {
    int deltaX = destX - curX;
    int deltaY = destY - curY;

    // Move along Y-axis (forward or backward)
    if (deltaY > 0) {
        forward(deltaY * CELL_TRAVERSE_TIME, 70);
        curY += deltaY;
    } else if (deltaY < 0) {
        reverse(deltaY * CELL_TRAVERSE_TIME, 70);
        curY -= deltaY;
    }

    stop(2000);

    // Adjust orientation and move along X-axis
    if (deltaX > 0) {
        turnRight(TURN_TIME_90, 60);
    } else if (deltaX < 0) {
        turnLeft(TURN_TIME_90, 60);
    }

    stop(500);

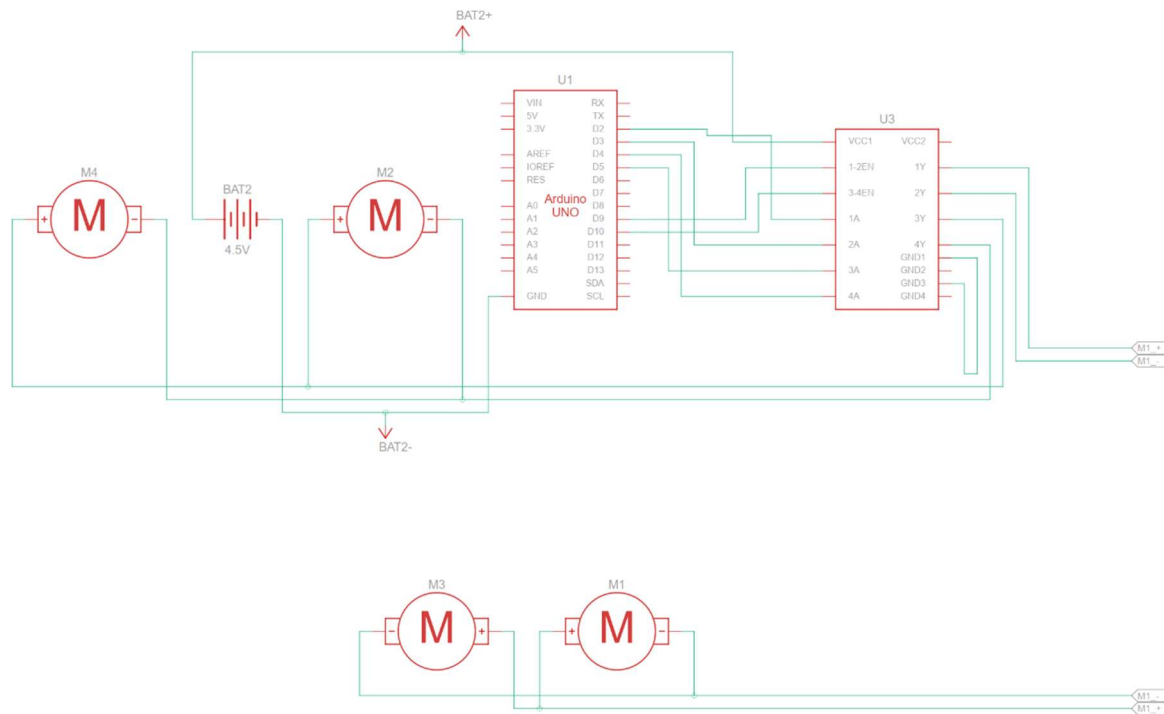
    forward(abs(deltaX) * CELL_TRAVERSE_TIME, 70);
    curX += deltaX;

    stop(2000);

    if (deltaX > 0) {
        turnLeft(TURN_TIME_90, 60);
    } else if (deltaX < 0) {
        turnRight(TURN_TIME_90, 60);
    }

    stop(2000);
}
```

2.4 Circuit Diagram



3. Implementation

3.1 Build and Assembly

1. Assembling the Chassis

The initial design used a 2WD chassis, where two BO motors were mounted onto the main frame using the provided brackets and screws. A trolley wheel was added at the front for balance. However, during testing, this configuration proved unstable and unsuitable for reliable grid navigation. To address this, the chassis was upgraded to a 4WD configuration by adding two additional BO motors to the front of the same chassis. These motors were secured using a glue gun, significantly improving the robot's balance and overall stability.

2. Mounting the Wheels

The 65mm motor wheels were securely press-fitted onto the BO motor shafts. The motor wires were routed carefully to prevent tangling or obstruction during motion. After the upgrade to 4WD, all four wheels contributed to smoother and more reliable movement across the grid.

3. Connecting the Motor Driver (L298N)

The motors were connected to the L298N motor driver module using the motor wires. These motor wires were connected in such a way that each pair of motors from a particular side (left or right) shared the same control signals, allowing us to use a single motor driver instead of two. After that, the Power Input Pins (IN1, IN2, IN3, IN4) and Control Pins (ENA, ENB) were connected to the Arduino.

4. Power Supply

A 4-cell AA battery holder was fixed to the underside of the chassis using a glue gun. An ON/OFF switch was installed in-line with the power connection to control the robot's operation. The system was powered using four Li-Po AA rechargeable batteries, which offered sufficient voltage and current for both the motors and control circuitry.

5. Breadboard and Wiring

A 170-point mini breadboard was used to organize the power connections, especially for distributing VCC and GND lines to various components. Jumper wires were used to interconnect all the modules, including the Arduino and Motor Driver. To maintain structural cleanliness and prevent loose wiring from interfering with movement, cable ties were used for proper cable management.

6. Component Placement

The Arduino Uno board and the L298N motor driver module were mounted on the top of the chassis using a glue gun. The 4xAA battery holder was attached to the underside of the chassis.

7. Uploading the Code

The Arduino was connected to a computer using a USB cable, and the final code was uploaded to the Arduino using the Arduino IDE.

3.2 Challenges faced and how they were resolved

During the building and testing phase, we faced several practical challenges that tested our problem-solving skills. The issues we encountered and how we dealt with them are as follows:

1. Chassis Limitation and Steering Mechanism Failure

What Happened: The initial robot design was based on a two-wheeled chassis, which made it difficult to steer effectively. An attempt was made to control direction by mounting a servo motor to steer a single trolley wheel. However, this configuration proved unreliable and unstable during movement.

Why: The servo-mounted trolley wheel required continuous power to maintain direction, which led to high battery consumption. Over time, the servo motor's performance degraded, resulting in inconsistent turning behavior. Additionally, the mechanical setup lacked the structural integrity needed to support precise steering and stable movement.

Solution: To address this issue, the chassis was upgraded to a 4-wheeled configuration by attaching two additional BO motors to ensure stability and even weight distribution. These additional motors were mounted at the front of the chassis, where a trolley wheel was originally placed. Rectangle-shaped holes were made in the chassis to insert supporting sticks that held the motors in place. The motors were then fixed securely using a glue gun for alignment and additional screws for strong mechanical support. This modification allowed the robot to move more reliably across the grid and eliminated the need for a separate steering mechanism.

2. Movement and Turning Inaccuracy

What Happened: Initial tests showed that the robot would consistently either go past or fall

short of the target cells and turn more or less than 90 degrees.

Why: The `CELL_TRAVERSE_TIME` and `TURN_TIME_90` constants were just initial guesses. The actual time required depends on motor speed, wheel diameter, surface friction, and battery voltage.

Solution: A precise calibration process was undertaken. The robot was commanded to move one cell or turn 90 degrees, and the resulting distance/angle was measured. The time constants in the code were then adjusted iteratively until the desired movement was achieved with acceptable accuracy on a specific test surface.

3. Inconsistent Performance Across Surfaces

What Happened: A robot calibrated on a smooth tile floor became highly inaccurate when moved to a carpeted surface.

Why: The change in surface friction significantly altered the relationship between motor run-time and distance traveled. The higher friction of the carpet caused the robot to move more slowly and cover less distance in the same amount of time.

Solution: For this project, the solution was to recalibrate the time constants specifically for the intended test surface. This highlights a major limitation of the system (calibration is environment-specific).

4. Performance Degradation Over Time

What Happened: The robot's accuracy noticeably decreased after several minutes of continuous operation. It began to undershoot its targets.

Why: As the AA batteries discharged, the voltage supplied to the motors dropped. Lower voltage results in lower motor speed, causing the robot to travel a shorter distance in the same calibrated time.

Solution: Fresh, fully charged batteries were used for each formal test run to ensure consistent power delivery. This confirmed that voltage stability is critical for time-based localization. A long-term solution would be to use a regulated power supply or a more stable battery chemistry (e.g., LiPo).

4. Testing and Results

4.1 Tests Performed

To evaluate the performance of the Time-Based Grid Localization Robot, a series of tests were conducted on a 5x5 grid of tiles, where each tile was 30cm x 30cm. The following tests were performed:

1. Straight-Line Accuracy Test

The destination was set to (0, 4) to test the robot's ability to travel a straight distance of four cells. The final position error was measured.

2. 90-Degree Turn Accuracy Test

The destination was set to (1, 0). This required no initial forward movement, only a single 90-

degree right turn followed by a one-cell forward movement. The accuracy of the turn was visually assessed, and the final position was measured.

3. Combined Movement Test

A more complex path to the destination (3, 4) was tested. This required the robot to travel four cells forward, turn right, and then travel three cells forward. This test assessed the effect of cumulative error.

4. Repeatability Test

The Combined Movement Test to (3, 4) was performed ten times in a row (resetting the robot to the origin each time) to evaluate the consistency of its performance.

4.2 Observations and Outcomes

The robot's performance was consistent with the expectations for a dead-reckoning system.

1. Straight-Line Test

The robot was highly accurate in straight-line travel, typically stopping within ± 3 cm of the target cell.

2. Turn Accuracy Test

The 90-degree turns were the largest source of error. The final angle was consistently off by an estimated ± 55 degrees. This error, while small for a single turn, was magnified in subsequent movements.

3. Combined Movement Test

The robot successfully reached the correct target cell (3, 4) in all trials. However, due to the cumulative error from the turn, its final position within the cell varied. The final error was typically around 5-10 cm from the center of the target cell.

4. Repeatability Test

The robot's behavior was highly repeatable. The final positions from the five trials were all clustered within a small area, demonstrating that while there was a systematic error, the system was not random. A slight decrease in distance traveled was noted in the last two trials, attributed to battery drain.

Table 1 Summary of Test Results

Test Name	Success Rate	Key Observation	Issues Noted
Straight-Line Accuracy	100%	Very low error in linear distance.	None.
90-Degree Turn	95%	Turn angle was mostly correct.	Slight angular error ($\pm 5^\circ$) was the main issue.

Combined Movement	100%	Reached the correct target cell every time.	Cumulative error from the turn affected the final position.
Repeatability	100%	Final positions were closely clustered.	Performance slightly degraded as batteries drained.

5. Conclusion and Future Scope

This project successfully achieved its objective of designing and building a Time-Based Grid Localization Robot. It effectively demonstrated the principles of autonomous navigation using dead reckoning in a controlled environment. The integration of the Arduino Uno, L298N motor driver, and a 4WD chassis, controlled by a time-based algorithm, resulted in a functional and cost-effective prototype. Testing confirmed that while the system is susceptible to cumulative errors, particularly from turns, it is capable of reaching a specified grid coordinate with reasonable accuracy and high repeatability. This project provided invaluable hands-on experience in robotics, open-loop control systems, and the critical importance of calibration.

The current robot serves as a robust foundation for numerous enhancements that would transition it from an open-loop to a closed-loop system, dramatically improving its accuracy and versatility. The most impactful upgrade would be the addition of rotary encoders to create a closed-loop control system, allowing for precise distance and angle measurements that eliminate the reliance on timed movements and correct for errors from battery drain or surface friction. Navigational accuracy could be further improved through sensor fusion, integrating an Inertial Measurement Unit (IMU) or a magnetometer to correct rotational drift. Moreover, the fixed path logic could be replaced with advanced pathfinding algorithms like A* or Dijkstra's, enabling the robot to navigate grids with obstacles. This system could be made fully interactive through dynamic mapping, where a Bluetooth or Wi-Fi module would allow the robot to receive new maps and coordinates wirelessly. These future enhancements would evolve the robot into a more intelligent and robust autonomous system suitable for a wider range of complex tasks.

6. GitHub Repository

<https://github.com/aqib-m31/Time-Based-Grid-Localization-Robot>